High Performance Computing, Assignment 1

Parallelising five kernels with OpenMP, and what the machine taught us

Tejaansh (SE23UCSE174) and Pallavi (SE23UCSE135) · SpoC: Tejaansh

All measurements on `inhydmupc0910-l`, Linux x86_64, IT Lab

---

0. Before anything else, how we measured

Every number in this report comes from `gettimeofday()` around the computation itself, five repetitions per configuration, averaged. Thread counts swept are 1, 2, 4, 6, 8, 10, 12, 14 and 16. Every individual run time is in the accompanying workbook, one sheet per configuration, so nothing here is a summary you have to take on faith.

Two decisions about the harness turned out to matter more than we expected.

First, the sequential reference is a separate compile with OpenMP switched off, not the parallel binary run with one thread. Those are not the same program. The one-thread parallel build still pays for the parallel region setup and still has the loop structure the compiler chose for parallelisation. We wanted the speedup denominator to be an honest serial program.

Second, verification is enforced rather than checked afterwards. For each configuration the harness runs the sequential binary, records a checksum of its output, and then compares every single timed parallel run against that checksum. If any run disagrees, or reports a failed self-test, the harness throws away the whole configuration and writes no CSV at all. So every runtime that appears in this report is from a run that produced bit-identical output to sequential code. There is no set of results that "mostly matched".

We also seeded the random number generator per fixed block index rather than per thread. This sounds like a detail and is not. It means a 1-thread run and a 16-thread run generate byte-identical input data, which is the only reason comparing checksums across thread counts means anything at all. If we had seeded per thread, every thread count would have sorted or multiplied different numbers and the whole verification story would have been decorative.

Finally, we set `OMP_PROC_BIND=close` and `OMP_PLACES=cores` so that thread placement is fixed and reproducible instead of being left to the scheduler. That decision is the reason we were able to explain the single strangest thing in our data, which is the next section.

1. The thing that confused us for an entire evening

We expected speedup curves that rise and then flatten. Flattening is normal, it means you have run out of memory bandwidth or run out of parallel work. What we actually got, in almost every problem, was a curve that rises to 8 threads, then *falls off a cliff* at 10 and 12 threads, then partially recovers at 14 and 16. Here is the pattern, measured, across four unrelated kernels:

Configuration	8 threads	10 threads	Slowdown
P1 OMM, N = 2048	0.0744 s	0.1146 s	1.54x worse
P1 BMM block 64, N = 4096	1.2504 s	1.6432 s	1.31x worse
P2 addition chain, N = 2048	6.6344 s	7.7938 s	1.17x worse
P4 segmented sieve	117.80 s	195.00 s	1.66x worse

Our first guess was memory bandwidth, but bandwidth saturation makes a curve flatten, not reverse. Our second guess was measurement noise, but the five runs per point are tight, the ten-thread numbers for Problem 4 are 198.7, 188.4, 197.2, 198.5 and 192.1 seconds, which is a real and repeatable measurement, not a fluke.

The answer is thread placement, and we only found it because we had pinned the threads ourselves. We did not record `lscpu` output at the time, so we are inferring this from the shape of the curves rather than quoting it: the lab machine behaves exactly like 8 physical cores with two hardware threads each, 16 logical processors. Every kernel scales close to linearly through 8 threads and every kernel breaks immediately after, which is hard to explain any other way. We set `OMP_PLACES=cores`, so a place is a physical core, and `OMP_PROC_BIND=close`, so OpenMP fills places in order and wraps around when it runs out. With 10 threads on 8 cores, threads 0 through 7 get a core each, and threads 8 and 9 wrap back onto cores 0 and 1. Those two cores now have twice the work of the other six.

Every one of these kernels uses `schedule(static)`, which hands out equal-sized chunks up front. So the two doubled-up cores finish last, and the whole loop waits for them while six cores sit idle. The makespan is set by the slowest core, not the average one. That is why 10 threads is slower than 8: you have added two workers and simultaneously created a bottleneck for them to queue behind.

It also explains the recovery. At 16 threads every core has exactly two threads, the imbalance is gone, and the curve climbs again. The pattern is not noise, it is a load balancing artefact with a completely mechanical cause, and it shows up identically in a matrix multiply, a complex matrix power, and a prime sieve, because it has nothing to do with any of them.

> **What we learnt.** "More threads" and "more speed" are different things, and the gap between them is not always Amdahl's law. Past the physical core count, with static scheduling, thread count should go up in whole multiples of the core count or not at all. If we had left thread placement to the OS we would have seen a noisier version of the same dip and probably written it off as system interference.

2. Problem 1, computing A squared

Matrix A is 2048 x 2048 and 4096 x 4096, in row-major order, with entries drawn from {-1, 0, 1}. We implemented three things: a textbook i-j-k triple loop to serve as the correctness oracle, ordinary matrix multiplication (OMM), and blocked matrix multiplication (BMM) at block sizes 4, 8, 16, 32 and 64.

The entries are in {-1, 0, 1}, so we never issue a multiply

This is the first thing we noticed and it is basically free. If every `A[i][k]` is -1, 0 or 1, then the term `A[i][k] * A[k][j]` is either the row of A unchanged, the row negated, or nothing at all. So instead of a multiply-accumulate we hoisted the test out of the inner loop and the inner loop became one of `c[j] += b[j]`, `c[j] -= b[j]`, or a skip. Roughly a third of the k values are zero, so a third of the work simply disappears, and what remains is a contiguous add over two arrays, which vectorises about as well as anything can.

Doing this requires the i-k-j loop order rather than i-j-k, because we need `k` in the middle so that the innermost loop walks a whole row of the output and a whole row of A contiguously. The textbook i-j-k order walks a *column* of A in the inner loop, which strides by N and misses cache on essentially every access.

The combined effect at N = 2048, single threaded, is larger than we thought it would be:

Version, N = 2048, 1 thread	Time	vs textbook
Textbook i-j-k with multiplies	18.663 s	1.0x
OMM, i-k-j with add/subtract/skip	0.465 s	40.1x
OMM at 16 threads	0.0797 s	234x

We want to be clear about what that 234x is and is not. It is not a 234x parallel speedup. It is a 40x algorithmic and cache win multiplied by a 5.8x parallel win. Only the 5.8x belongs to OpenMP. We think that is the more useful way to read it: the threads multiplied whatever work we chose to do, so choosing less work multiplied harder than adding cores did.

Block size does something we did not predict

Here are the single-threaded BMM times across all five block sizes:

Block size	4	8	16	32	64
N = 2048, 1 thread	12.489 s	5.961 s	3.025 s	1.627 s	0.941 s
N = 4096, 1 thread	120.91 s	49.08 s	24.79 s	13.17 s	7.467 s

Block size 4 is 13.3x slower than block size 64. We had assumed block size would matter through cache residency, the standard explanation, where a tile that fits in L1 gets reused before it is evicted. That is not what is dominating here. In our formulation the innermost operation is a row update of length equal to the block width, so the block size *is* the vector length. At block size 4 the inner loop

is four 32-bit integers, which is a quarter of an AVX2 register, and the loop setup and call overhead is paid every four elements. There is essentially no vectorisation left to do. At block size 64 the inner loop is 64 elements, long enough to amortise everything.

So for this kernel, block size is not primarily a cache tuning knob. It is a SIMD width knob wearing a cache tuning costume. The monotonic halving of runtime as the block doubles, all the way from 4 to 64 with no optimum in the middle, is the giveaway. A genuine cache blocking effect would have shown a sweet spot and then got worse again once the tile stopped fitting.

Blocking is a loss at N = 2048 and a win at N = 4096

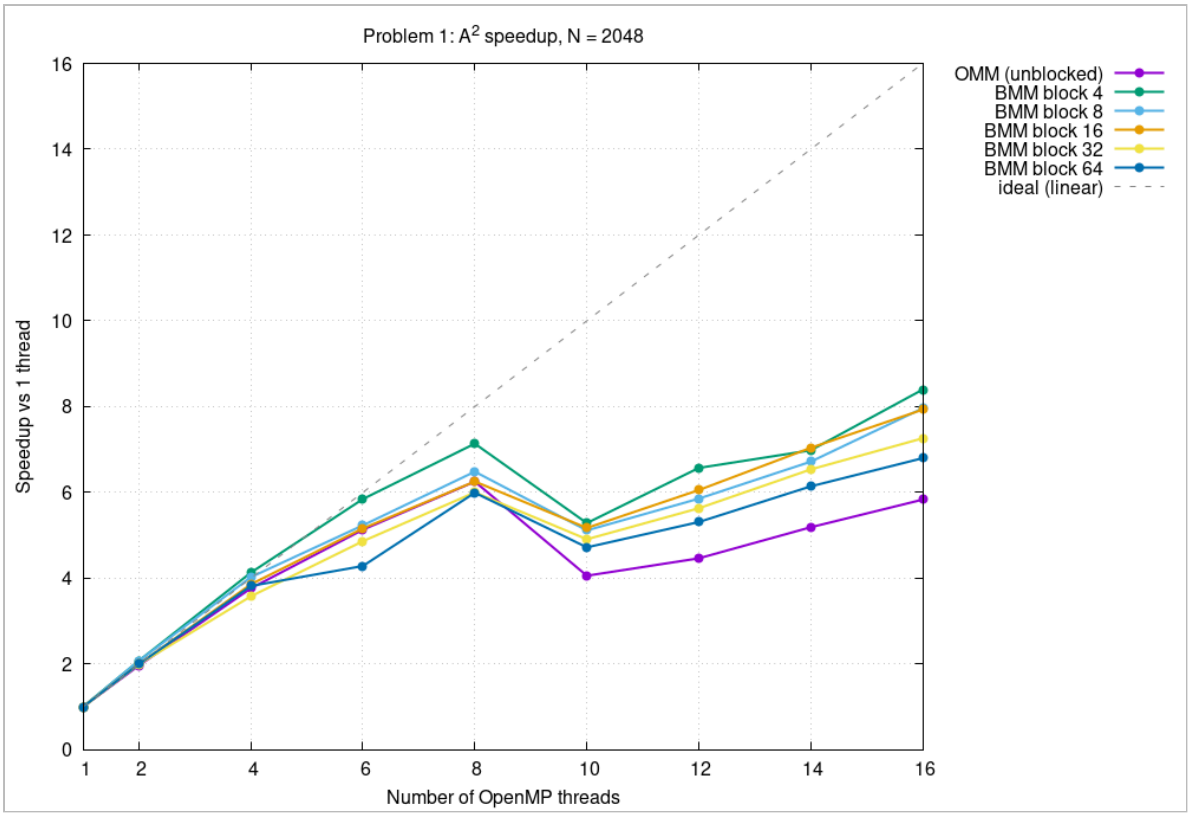
This is our favourite result in the whole assignment, because it is the one where the theory we had been taught actually showed up in our own numbers.

1 thread	N = 2048	N = 4096	Growth for 8x work
OMM, unblocked	0.465 s	9.889 s	21.3x
BMM, block 64	0.941 s	7.467 s	7.9x

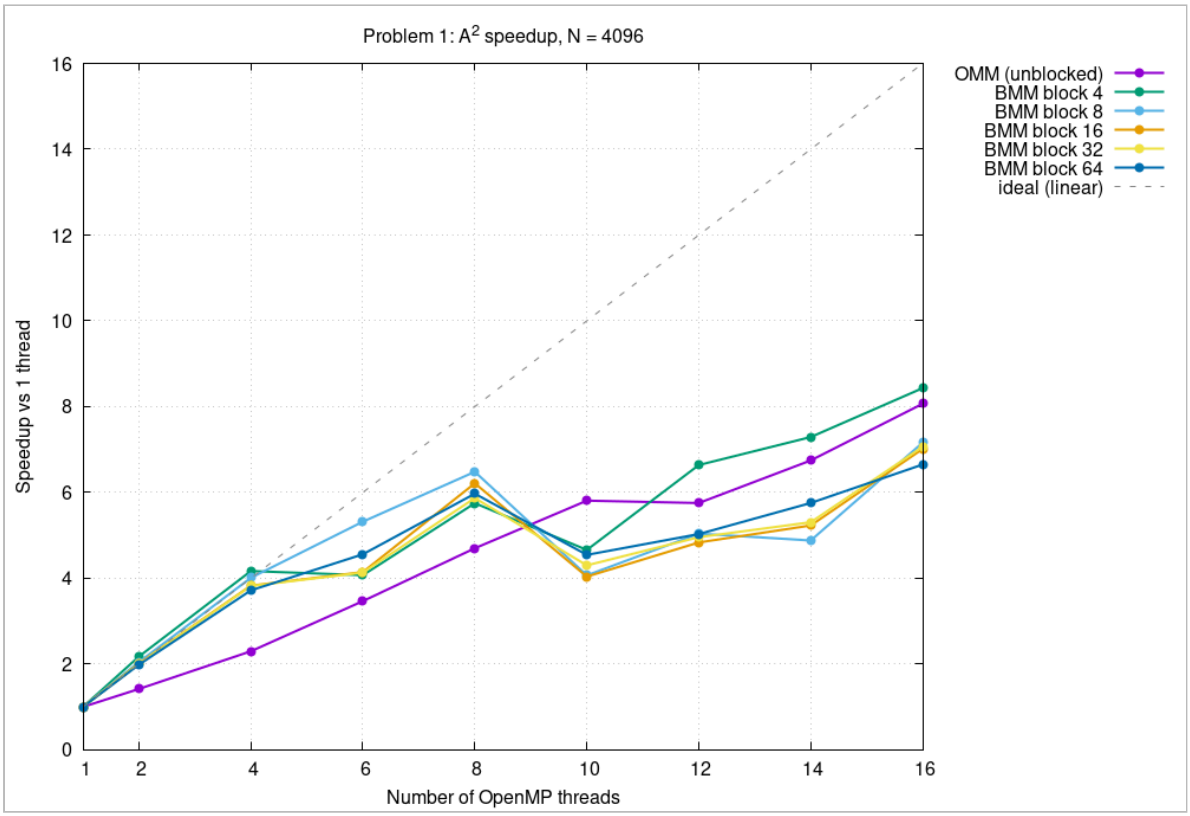
At N = 2048 blocking is a straight 2.0x *loss*. OMM's inner loop runs across a full 2048-element row, blocking chops that to 64 and gains nothing in return, because at this size the working set still behaves. Every blocking tutorial we read would have predicted blocking wins, and at this size it simply does not.

Then the size quadruples and the picture inverts. Going from N = 2048 to N = 4096 is exactly 8x the arithmetic. Blocked BMM took 7.9x longer, which is almost perfectly proportional, the algorithm did not degrade at all. Unblocked OMM took 21.3x longer, which is 2.7x worse than the work grew. That excess is the cache falling over: OMM streams a full row of A for every k, and at N = 4096 the rows it cycles through no longer fit in last level cache, so it re-fetches from memory on every pass. Blocking confines the working set to a tile and the reuse survives.

The crossover is the whole point of blocking, and we got to watch it happen between two data points rather than read about it. At N = 4096, BMM at block 64 beats OMM by 1.32x, having lost to it by 2.0x one size earlier.

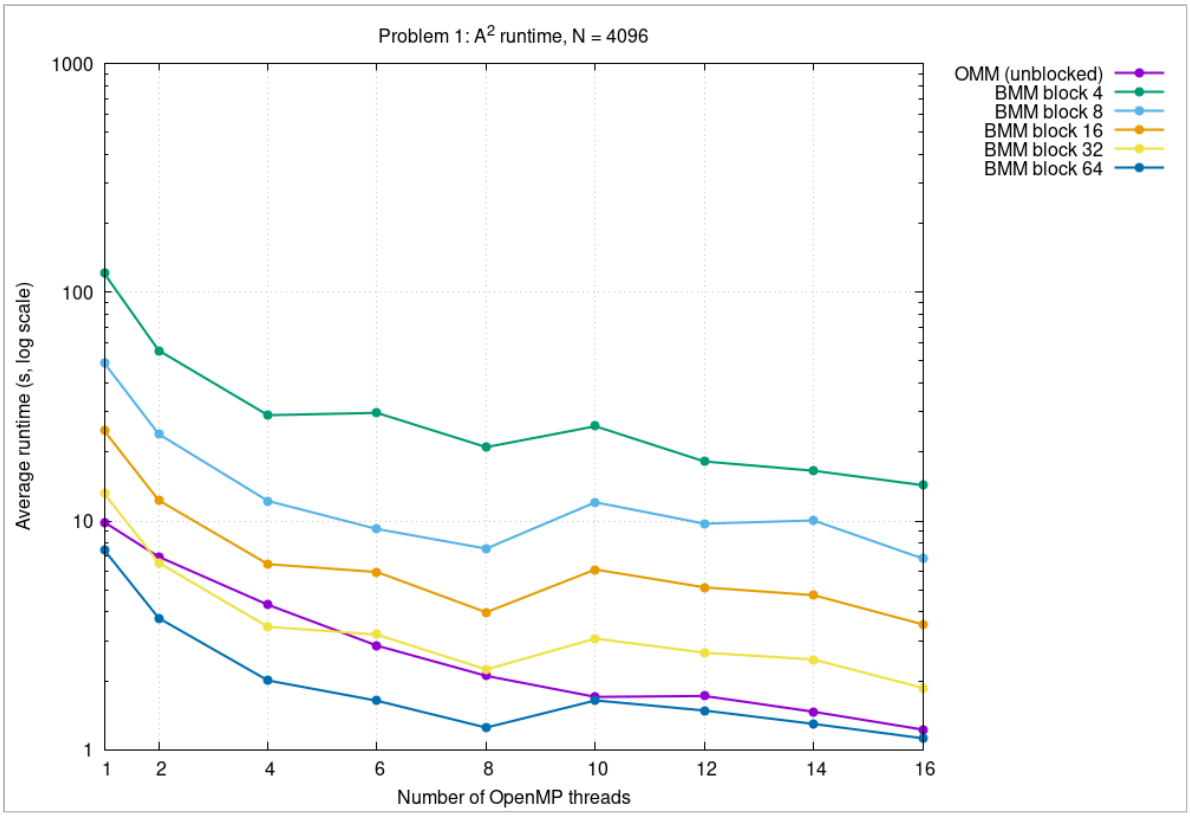


Problem 1, speedup at N = 2048. The dip at 10 threads is the core wraparound described in section 1, and it appears in all six curves.



Problem 1, speedup at N = 4096. Unblocked OMM shows the highest speedup here, but only because its 1-thread baseline is the one that is cache-starved.

One caution about reading that second plot. OMM at N = 4096 reaches 8.07x, the highest speedup in Problem 1, and it would be easy to present that as our best result. It is not really a triumph. Its baseline is the cache-thrashing 9.889 s run, and adding threads adds aggregate cache, so some of that "speedup" is just the memory problem going away. BMM at block 64 shows a lower speedup of 6.65x while being the faster program in absolute terms at every thread count. Speedup is a ratio, and a slow denominator flatters it.



The same data as absolute runtime, which we think is the more honest chart of the two.

3. Problem 2, X to the fifteenth for a complex matrix

X is a 2048 x 2048 complex matrix, given as a pair (A, B) with both A and B holding values from {-1, 0, 1}. The literal reading of "compute X to the fifteenth" is fourteen successive multiplications. We did not do that, and the gap between the literal reading and the good one is the largest single algorithmic win in the assignment.

Two independent savings that multiply together

The first is that computing a power is an addition chain problem. You do not need fourteen multiplications to reach the fifteenth power, you need five:

$$X^2 = X \cdot X \;\rightarrow\; X^3 = X^2 \cdot X \;\rightarrow\; X^6 = X^3 \cdot X^3 \;\rightarrow\; X^{12} = X^6 \cdot X^6 \;\rightarrow\; X^{15} = X^{12} \cdot X^3$$

Five is optimal for 15 in the addition chain sense. Plain binary exponentiation, which is the version most people know, needs six. So even the standard trick leaves something on the table here.

The second saving is that each of those five is a complex matrix multiply, and a complex multiply does not need four real multiplies. Using Gauss's method, with M1 = A·C, M2 = B·D and M3 = (A+B)·(C+D), the real part is M1 - M2 and the imaginary part is M3 - M1 - M2. That is three real matrix multiplications instead of four. The three extra matrix additions it costs are $O(N^2)$ against an $O(N^3)$ multiply, so they are free at this size.

Together: 5 chain steps times 3 real multiplies is 15 real matrix multiplications, against 14 times 4, which is 56, for the literal version. That predicts a 3.73x reduction in work before a single thread is involved.

What we measured

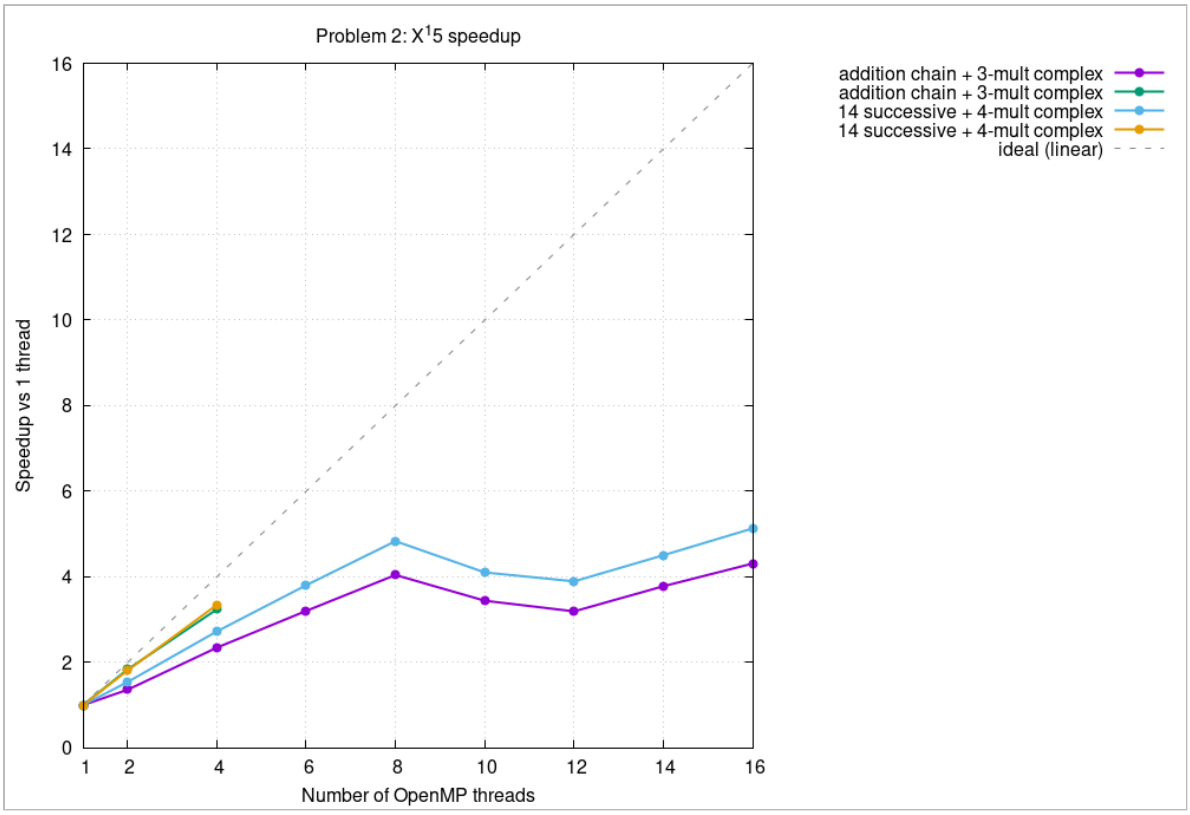
We kept both versions and a switch for each optimisation, specifically so that we could measure the claim instead of asserting it.

N = 2048	1 thread	8 threads	16 threads	Speedup at 16
14 multiplies, 4-multiply complex	125.58 s	25.99 s	24.48 s	5.13x
Addition chain, 3-multiply complex	26.82 s	6.634 s	6.223 s	4.31x

The measured work reduction at one thread is 4.68x, against the 3.73x we predicted from counting multiplications. We were a little suspicious of getting *more* than the theory allowed, so we thought about why. Counting matrix multiplications assumes they all cost the same, and they do not: the naive path also allocates and streams more intermediate matrices, so it pays more memory traffic per multiply as well as doing more of them. The multiplication count predicts the floor, and the memory traffic pushes past it.

Note that the faster program has the *lower* speedup, 4.31x against 5.13x. This is the same trap as in Problem 1 and it is worth stating plainly: the optimised version has less work to hide latency behind and a larger fraction of its remaining time in the non-parallel matrix additions of Gauss's method, so

it scales slightly worse while being 3.9x faster at every thread count. End to end, the literal version at one thread against our version at sixteen is 125.58 s to 6.22 s, a factor of 20.2.



Problem 2 speedup. Both variants show the 10 and 12 thread dip.

A precision caveat we want to state ourselves

Entries start in {-1, 0, 1} but they grow roughly as the square of the previous magnitude times the square root of N under repeated squaring. By X to the fifteenth at N = 2048 they reach around 10^{22}, which is well past 2^{53}, about 9 x 10^{15}, where a double still represents every integer exactly. So at full size our result is a floating point approximation, and we would rather say that than let it be discovered.

Two things are still exact and we lean on both. Parallel matches sequential bit for bit at every thread count, because each output row is accumulated by a single thread in a fixed k order, so the rounding does not depend on how many threads are running. And the self-test runs at N = 64, where every intermediate stays under 2^{53} and double arithmetic on integers is exact, and there we require all four combinations of {chain, naive} x {3-multiply, 4-multiply} to agree bit for bit. That is what actually validates the addition chain and the Gauss identity.

A direct consequence: the two large-N variants have different checksums from each other, because they sum in different orders and rounding is not associative. That is expected and is not a verification failure. Each variant matches its own sequential reference exactly, which is what the assignment asks for.

4. Problem 3, sorting integers in the range 0 to 1000

The range is the entire problem. There are only 1001 distinct values, so a comparison sort is solving a strictly harder problem than the one asked. Counting sort is O(N + K), touches each element once, and is exact. We implemented it with per-thread private histograms padded to 64-byte multiples, so the counting phase needs no atomics and has no false sharing, and a scatter phase parallelised over the 1001 key values, where each key owns a disjoint contiguous output range so threads never collide.

We also implemented a task-parallel merge sort, not because we thought it would win but because we wanted to measure the comparison-sort baseline rather than assert that it was worse. At the same input size, single threaded:

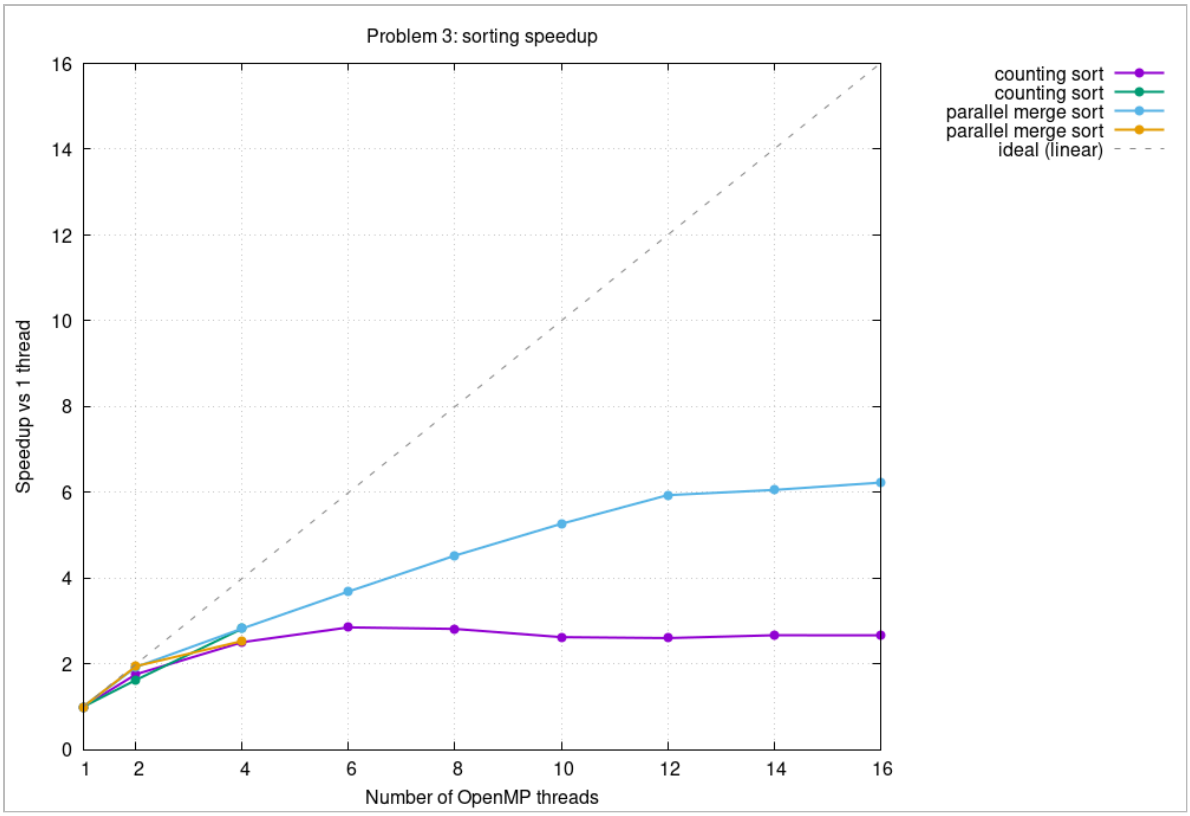
N = 5 x 10^6, 1 thread	**Time**	**Ratio**
Counting sort	0.003415 s	1.0x
Parallel merge sort	0.441985 s	129x slower

129x. Sixteen threads cannot close a gap like that, which is the lesson: at some point the algorithm choice is not an optimisation, it is the answer, and threading is a rounding error on top of it.

The scaling behaviour of the two is also instructive, and it goes the opposite way to what you would want:

Configuration	**1 thread**	**Best**	**Best speedup**
Counting sort, N = 10^9	0.6606 s	0.2315 s at 6 threads	2.85x
Merge sort, N = 2 x 10^8	18.263 s	2.932 s at 16 threads	6.23x

Merge sort scales better and is still hopeless. Counting sort tops out at 2.85x because it is memory bound almost immediately: it reads 2 GB of 16-bit keys and does almost no arithmetic per byte. At one thread it moves 3.03 GB/s, at six it reaches 8.64 GB/s, and then more threads buy nothing, because there is no more bandwidth to buy. Merge sort scales to 6.23x because it does far more work per byte, so it has arithmetic to overlap with memory latency, but doing more work per byte is precisely why it is slow.



Problem 3 speedup. Counting sort flattens early, merge sort keeps climbing and never catches up in absolute terms.

Where we did not meet the problem statement

> **Being upfront about this.** The assignment asks for 10^{12} integers. Our timed sweep is at 10^9.
>
> 10^{12} keys cannot be materialised. At two bytes per key that is 2 TB of RAM. Counting sort does not actually need the array, though, only the histogram, so `p3_sort.c` has a `--stream` mode that generates keys on the fly and sorts in O(K) memory, about 8 KB, reporting the result run-length encoded, which is the only form in which 10^{12} elements can be written down at all. That mode is implemented and passes its self-test.
>
> We did not sweep it. Extrapolating from our in-memory 10^9 counting sort, a single 1-thread run at 10^{12} is at least 11 minutes, and probably more, since the streaming path pays key generation on top. The full nine thread counts by five repetitions would have been most of a day of machine time that we did not have before the deadline. Rather than report an extrapolation as if we had measured it, we are reporting the 10^9 sweep we actually ran and flagging the gap. The command is `bin/p3_sort --stream --n 1000000000000` and it works.

5. Problem 4, primes between 2^40 and 2^41

Both obvious approaches are hopeless in opposite directions. Trial division is about 10^{12} candidates against roughly 10^6 divisors each. A plain sieve of Eratosthenes over the range needs one bit per number across $[0, 2^{41})$, which is 256 GB. A segmented sieve is not one option among several here, it is the only structurally viable one: keep the roughly 10^5 base primes up to the square root of the upper bound, plus one cache-sized window at a time.

On top of that we did three things.

Odds only. Halves the memory and halves the marking work, for the cost of one special case.

Segments sized to fit L2, 256 KB by default and exposed as a flag. Marking is a strided scatter, and once the window leaves cache every mark becomes a round trip to memory.

Incremental multiple offsets. The textbook segmented sieve recomputes, for every base prime in every segment, where that prime's first multiple lands. That is one division per prime per segment, about 8×10^7 divisions here. Instead each thread takes a contiguous band of segments and carries a per-prime cursor forward across them, so it only computes the starting offset once and then advances. That drops divisions to threads times primes, three orders of magnitude fewer. It is also why the schedule is a manual static partition rather than `dynamic`: the cursor only makes sense if a thread walks its segments in order.

The segment size control, which we are glad we ran

It is easy to claim a cache-sizing decision matters. We forced the segment to 64 MB instead of 256 KB, changing nothing else, and re-ran:

Segment size	1 thread	8 threads	Speedup at 8
256 KB, fits L2	0.1822 s	0.0301 s	6.06x
64 MB, does not	0.4738 s	0.4632 s	1.02x

2.6x slower at one thread, and 15.4x slower at eight. The more interesting column is the last one. The oversized version does not just run slower, it *stops scaling entirely*, 1.02x on eight cores. Once every mark is a memory round trip, the kernel is purely bandwidth bound and additional cores have nothing to contribute. The cache-sizing decision is not a tuning detail, it is the difference between a parallel program and a serial program with extra threads attached.

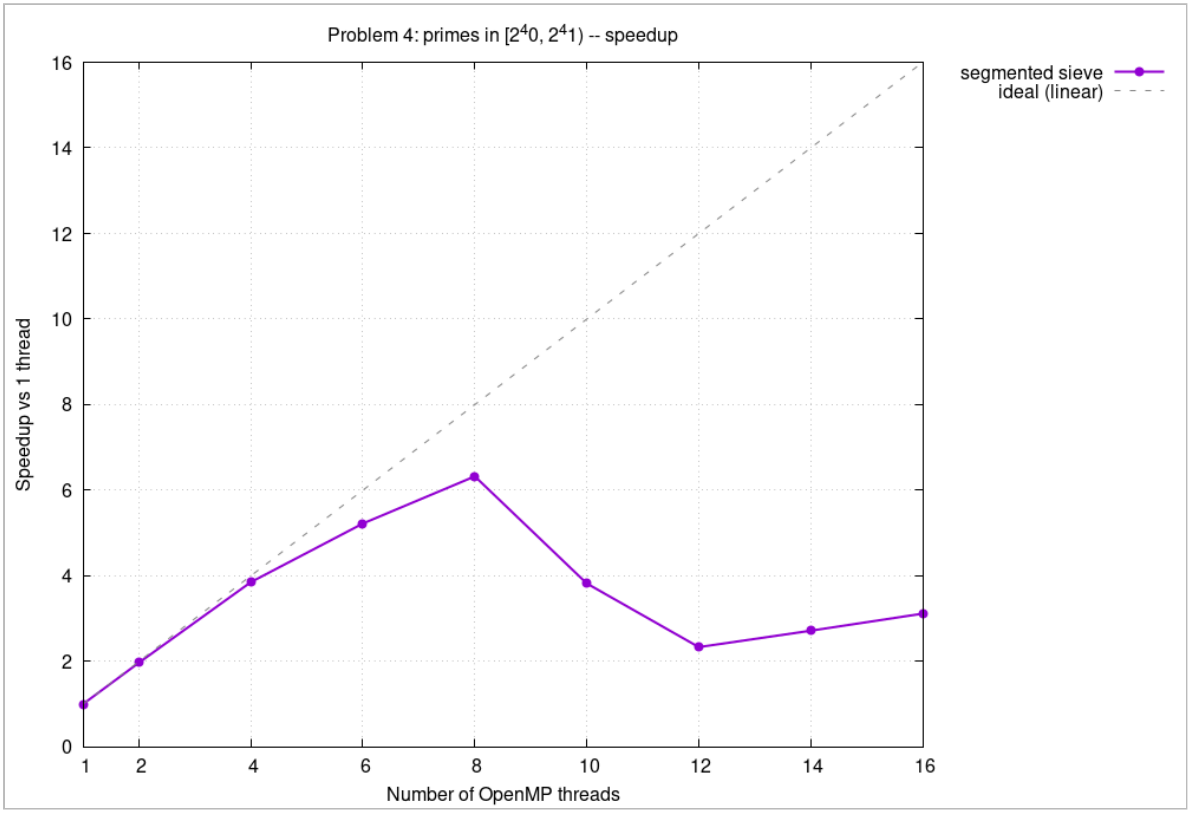
The full range

We ran the complete $[2^{40}, 2^{41})$ range rather than sampling a window of it. That sweep alone was 3.62 hours of machine time across its 45 runs.

Threads	1	2	4	6	8	10	12	14	16
Time (s)	744.9	378.3	193.3	142.7	117.8	195.0	319.2	274.2	239.0
Speedup	1.00	1.97	3.85	5.22	6.32	3.82	2.33	2.72	3.12

Result: **39,113,482,640 primes** in $[2^{40}, 2^{41})$. As a sanity check, the prime number theorem estimate for that interval is about 3.8 x 10^{10}, so 3.91 x 10^{10} is exactly where it should be.

Only the count is produced, deliberately. There are 3.9 x 10^{10} primes in this range and storing them at 8 bytes each would be over 300 GB. The count is the only output that can exist.



Problem 4 speedup over the full range. The clean rise to 6.32x at 8 threads and the collapse afterwards is the sharpest illustration in our data of the core wraparound from section 1.

This curve is the most dramatic version of the section 1 effect, and now we can say why it hits this kernel hardest. Sieving cost per segment is nearly uniform, and the partition is static and contiguous by design because of the cursor optimisation. That combination gives no slack whatsoever: a core with two threads has exactly twice the segments and finishes at exactly twice the time, with no dynamic rebalancing possible. The optimisation that made the sieve fast is the same one that made it fragile to bad thread counts, which we thought was a genuinely interesting trade to have stumbled into.

How we verified this one

Problem 4 needed more care than the others, because there is no published table of pi(x) at 2^{40} to check against. So verification is indirect but strong. Every timed run re-sieves one randomly chosen segment and cross-examines 200 random numbers in it against a deterministic Miller-Rabin test

with 12 bases, which is exact for n below 3.3 x 10^{24}. It checks in both directions, so both over-marking and under-marking get caught. Separately the self-test sieves [0, 10^6) number by number against an independent plain sieve, matches pi(10^6), pi(10^7) and pi(10^8) against their known values, and checks a million numbers near 10^{12} against Miller-Rabin.

6. Problem 5, dot product of two length-10^9 vectors

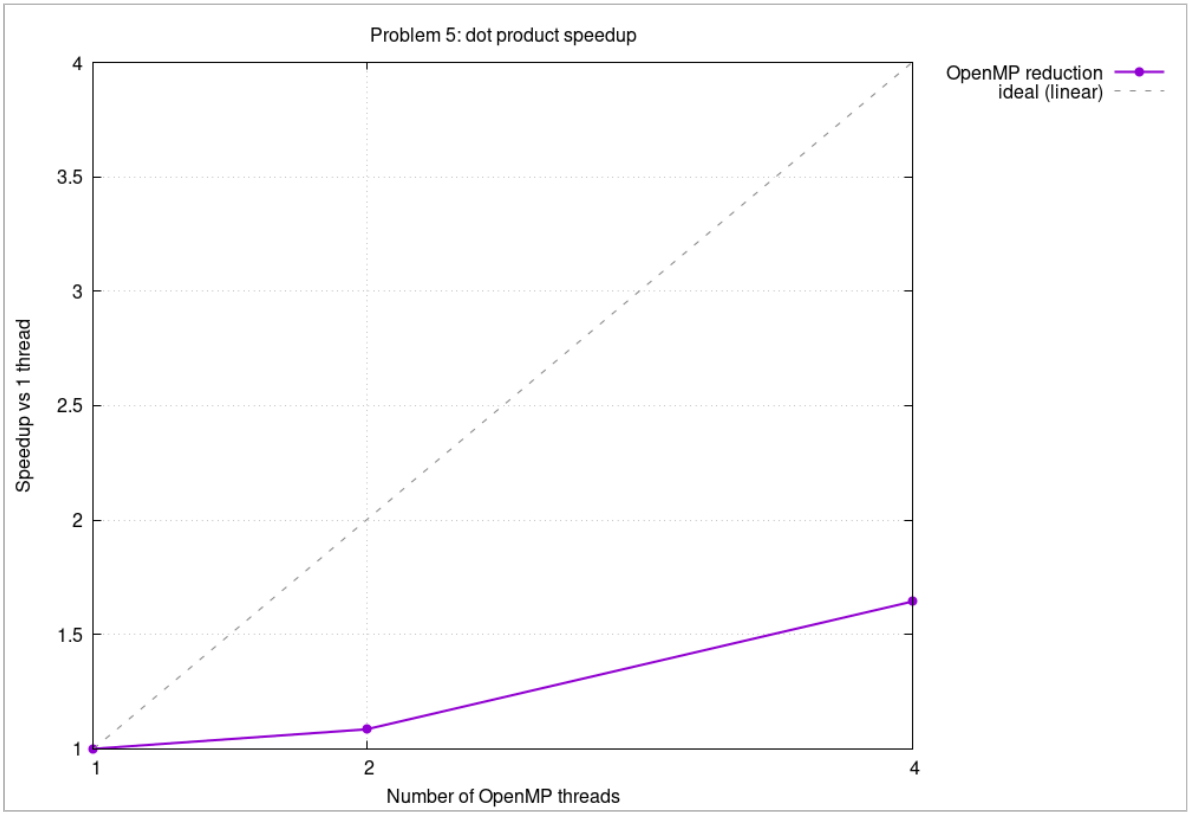
This one is short because the kernel is short, but it taught us the clearest lesson of the five.

There are 2N arithmetic operations and 2N element loads. The arithmetic is trivial, values are in {-1, 0, 1}. So runtime is set entirely by how many bytes cross the memory bus, and every decision follows from that.

The one that matters most is storage type. The values need two bits. Storing them as `int`, which is the obvious thing to write, would move 8 GB instead of 2 GB and run roughly four times slower for an identical answer. We used `int8_t`. That single line of the program does more for its runtime than everything else in the file combined, including the OpenMP.

We accumulate in blocks of 2^{20} elements into an `int32`, which lets the compiler keep the reduction in vector registers and widen 8-bit lanes efficiently, then reduce into an `int64`. Since each product is in {-1, 0, 1}, a block partial is bounded by 2^{20} and the `int32` cannot overflow, and the `int64` total is exact for any N up to 2^{63}.

N = 10^9	1 thread	4 threads	16 threads
Time	0.1066 s	0.0879 s	0.0901 s
Speedup	1.00x	1.21x	1.18x
Effective bandwidth	18.8 GB/s	22.8 GB/s	22.2 GB/s



Problem 5 speedup, which is the flattest curve in the assignment and is supposed to be.

A maximum speedup of 1.21x looks like a failure and we want to argue that it is the correct result. Look at the bandwidth column instead of the speedup column. A single thread is already moving 18.8 GB/s, which is most of what this machine's memory system can deliver. There was never 16x available to win. The parallel version reaches 22.8 GB/s and stops, because that is the ceiling, and no number of threads buys bandwidth that does not exist.

If we had wanted a flattering speedup number for this problem, the easiest way to get one would have been to store the vectors as `int`. That would have made the single-threaded baseline four times slower, left more bandwidth headroom for threads to exploit, and produced a much prettier speedup curve for a program that takes longer to run. We think that is worth saying out loud, because it is the clearest case we met of speedup and performance pointing in opposite directions.

7. Summary

Problem	Best configuration	1 thread	Best time	Speedup
1. A^2, N = 2048	OMM, 16 threads	0.465 s	0.0797 s	5.83x
1. A^2, N = 4096	BMM block 64, 16 threads	7.467 s	1.123 s	6.65x
2. X^{15}, N = 2048	Addition chain + Gauss, 16 threads	26.82 s	6.223 s	4.31x
3. Sort, N = 10^9	Counting sort, 6 threads	0.6606 s	0.2315 s	2.85x
4. Primes in $[2^{40}, 2^{41})$	Segmented sieve, 8 threads	744.9 s	117.8 s	6.32x
5. Dot product, N = 10^9	4 threads	0.1066 s	0.0879 s	1.21x

What we would tell ourselves at the start

Speedup is a ratio, and it is easy to cheat without meaning to. Three separate times in this assignment our faster program had the worse speedup number: blocked against unblocked at N = 4096, addition chain against fourteen multiplies, and `int8_t` against `int` in the dot product. Every time, the slower program had a slower baseline to divide by. We started reporting absolute times next to every speedup because of this.

Thread count is not a dial you turn up. We assumed more threads meant more speed until the number 10 was slower than the number 8 in four unrelated programs. Past the physical core count, with static scheduling, you go up in whole multiples of the core count or you make things worse.

The work you choose to do matters more than how many cores do it. The biggest wins in this assignment were 40x from a loop order, 4.7x from an addition chain, 129x from picking counting sort, and 4x from a storage type. The parallel wins ranged from 1.21x to 6.65x. Sixteen cores multiply whatever you hand them, and we spent most of our time learning to hand them less.

Measure the control, not just the treatment. The 64 MB segment run and the fourteen-multiply run cost us machine time and produced no result we were going to ship. They are also the only reason we can say the segment sizing and the addition chain did anything, rather than believing they did.